

Practice Set: Classes and Objects in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define a **Class** and an **Object** in C++. Explain why an instantiated instance of a class is specifically called an "object" rather than merely a scalar variable.

Question 2: Explain the fundamental technical difference between a **class** and a **struct** in C++ regarding default access specifiers and data security.

Question 3: Detail how classes and objects are represented in physical RAM. How much memory does a class blueprint consume, what is stored inside an object's stack allocation, and where do member functions reside in memory?

Question 4: Draw or map the memory layout of two instances (c1 and c2) of a Complex class containing two integer data members (int a, b;). Explain how both instances interact with the setData() function stored in the Code/Text segment.

Question 5: Explain the syntax and purpose of the **Scope Resolution Operator (::)** when defining a member function outside the class body. Break down each component of a membership label (e.g., void Complex::showData()).

Question 6: What is the compiler optimization rule regarding member functions defined directly inside a class body versus those defined outside using ::? Why should large functions be defined outside?

Question 7: Why can't standard arithmetic operators like + or - operate directly on user-defined objects without operator overloading? How do member functions taking and returning objects resolve this limitation?

Question 8: In the statement c3 = c1.add(c2);, contrast the **Caller Object (c1)** and the **Argument Object (c2)** in terms of invocation role, member variable access syntax, and internal compiler binding mechanisms.

Question 9: Explain the common beginner gotcha of referencing caller objects by identifier name (e.g., writing `temp.a = c1.a + c2.a;`) inside a member function like `Complex::add(Complex c)`. Why does the compiler reject this?

Question 10: Write a complete C++ program defining a `Complex` class with private data members `a` and `b`, a constructor/setter, a `showData()` method defined outside using `::`, and an `add()` member function that accepts an object by const reference and returns the sum as a new object.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): How many bytes of RAM are allocated when a class definition header is parsed by the compiler before any object is created?

Question 12 (Viva): Are member functions duplicated in RAM for every instantiated object of a class?

Question 13 (Viva): Through which hidden pointer does a member function know which specific object invoked it?

Question 14 (Viva): What is the default access specifier for variables and functions declared in a C++ class if no label is specified?

Question 15 (Viva): What is the default access specifier in a C++ struct?

Question 16 (Viva): What keyword/operator links a member function defined outside a class back to its class blueprint?

Question 17 (Viva): What happens to member functions defined directly inside the class definition in terms of inline expansion?

Question 18 (Viva): Why is passing objects by **Const Reference** (`const ClassName &obj`) considered best practice in member functions?

Question 19 (Viva): If an object `c1` has two `int` members, what is the value of `sizeof(c1)` on a standard 32-bit/64-bit architecture?

Question 20 (Viva): Where is the machine executable code of class member functions stored in memory?

Answers & Explanations

Answer: 01

Class: A user-defined blueprint or schema that encapsulates data members (state) and member functions (behavior) into a single entity. It serves as an abstract custom data type.

Object: A concrete runtime instance of a class. Unlike primitive scalar variables that only hold raw values, an object possesses encapsulated state, behavior, and identity, interacting via member methods.

Answer: 02

In C++, class and struct are functionally equivalent with identical capabilities (methods, constructors, inheritance).

The Difference: In a struct, all members default to **public**. In a class, all members default to **private**, enforcing data hiding and encapsulation by default.

Answer: 03

Class Definition: Allocates **0 bytes** of physical RAM at runtime; it is purely a compiler type schema.

Object Instantiation: Allocates memory on the Stack or Heap *only* for the object's non-static data members.

Member Functions: Loaded once into the read-only **Code/Text Segment**. All instances share the exact same compiled function instructions.

Answer: 04

Memory Map:

c1 Stack Block (8 Bytes at 0x7ffd10): int a (4 Bytes) + int b (4 Bytes).

c2 Stack Block (8 Bytes at 0x7ffd20): int a (4 Bytes) + int b (4 Bytes).

Interaction: When c1.setData(3, 4) or c2.setData(5, 6) executes, CPU control jumps to the single routine located in the Code Segment (0x00401240). The compiler passes the caller's address via the implicit this pointer so the function modifies the correct memory block.

Answer: 05

Purpose: The Scope Resolution Operator (::) establishes membership, informing the compiler that an externally written function definition belongs to a specific class blueprint.

Deconstruction of void Complex::showData():

void: Return type of the function.

Complex: Class name to which the function belongs.

::: Scope resolution operator (membership label).

showData(): Function identifier and parameter list.

Answer: 06

Implicit Inline: Any member function defined inside the class body is treated by the compiler as **implicitly inline**.

Defining Outside: Defining large, complex functions inside the class can cause binary code bloat across translation units. Defining them outside using :: ensures the function is compiled once as a standard routine in the code segment.

Answer: 07

Limitation: Standard operators (+, -) have native hardware instructions only for primitive data types (int, float). The compiler cannot infer how to aggregate custom members of user-defined types without explicit operator overloading.

Solution: A member function can accept another object as an argument, calculate member-wise sums into a temporary object, and return that object to the caller.

Answer: 08

Feature	Caller Object (c1)	Argument Object (c2)
Invocation Role	Invokes the method (c1.add(...)).	Passed inside parentheses as parameter.
Member Access	Direct access without dot operator (a, b).	Requires dot operator on parameter (c.a, c.b).
Compiler Binding	Bound via the implicit this pointer.	Passed as standard parameter on call stack.

Answer: 09

Identifiers like c1 and c2 are local variables declared inside main() and are completely out of scope inside Complex::add(). The function must access caller

members directly (via this->a or simply a) and argument members through the parameter identifier (e.g., c.a).

Answer: 10

```
#include <iostream>
using namespace std;
class Complex {private:
    int a, b;public:
    void setData(int x, int y)
    {
        a = x;
        b = y;
    }

    void showData(); // Declaration

    // Member function accepting const reference and returning an object
    Complex add(const Complex &c) const
    {
        Complex temp;
        temp.a = a + c.a; // 'a' belongs to caller, 'c.a' to argument
        temp.b = b + c.b;
        return temp;
    }
};
// Defining member function outside class using ::void Complex::showData() {
    cout << "Real: " << a << " | Imaginary: " << b << endl;
}
int main()
{
    Complex c1, c2, c3;
    c1.setData(3, 4);
    c2.setData(5, 6);

    c3 = c1.add(c2);

    cout << "Result of addition in c3:" << endl;
    c3.showData(); // Outputs: Real: 8 | Imaginary: 10

    return 0;
}
```

Answer: 11

0 bytes; class definitions serve solely as type templates during compilation.

Answer: 12

No; member functions are compiled once into the Code/Text segment and shared across all instances.

Answer: 13

The **this** pointer (a hidden pointer holding the invoking object's memory address).

Answer: 14

private.

Answer: 15

public.

Answer: 16

The **Scope Resolution Operator (::)**.

Answer: 17

They are automatically treated as **implicitly inline**.

Answer: 18

It prevents expensive stack copy allocations of object data while guaranteeing that the passed object cannot be modified.

Answer: 19

8 bytes (4 bytes for int a + 4 bytes for int b).

Answer: 20

In the **Code / Text Segment** of memory.